# CEC Quantum Computing

## Lecture 7: Quantum Machine Learning, Kernels, and Hybrid Models

Dr. Pavel Sulimov  Prof. Dr. Rudolf Marcel Füchslin  Claude Lehmann

April 8, 2026

ZHAW School of Engineering, Continuing Education

# Motivation

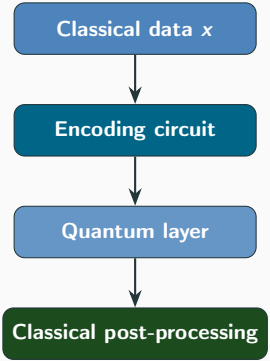
# What Do We Mean by Quantum Machine Learning?

## Three meanings in practice

- **Quantum data + quantum model**: rare today
- **Classical data + quantum model**: most near-term QML papers
- **Hybrid quantum-classical model**: the realistic NISQ setting

## Main message

For this course, **QML mostly means hybrid models**: classical data, a small quantum circuit, and a classical optimiser or neural network around it.



# Why Not Expect Magic?

- A quantum circuit can create **unusual feature maps**, but that does **not** imply generic speedup.
- Classical data must first be loaded into the circuit. Data loading can dominate the whole workflow.
- Some proposed QML advantages disappear after **dequantization**: a classical algorithm is later found that does something similar.
- The useful question is not *"Is quantum always better?"* but rather

  When does a quantum feature map match the data structure?

**Reasonable learning goal**

Understand **how** quantum circuits enter ML pipelines, what the trade-offs are, and where kernels or hybrid models may be worth trying.

# A Taxonomy for Beginners

| Setting | Input | Quantum role | Typical today? |
|---|---|---|---|
| Quantum data learning | quantum states | process/measure quantum data | No |
| Quantum kernels | classical vectors | estimate inner products/fidelities | Sometimes |
| Variational classifier | classical vectors | train a PQC end-to-end | Yes |
| Hybrid QNN | classical vectors | quantum layer inside a bigger model | Yes |

## Lecture logic

Today we move in this order: **encoding** → **feature maps** → **kernels** → **hybrid training** → **nonlinearity**.

# Encoding

# Encoding: classical data into qubits

**Topic.** Load classical features into qubit phases, amplitudes, or basis labels before the rest of the circuit.

**Common mistake:** treat encoding as simple preprocessing.

**Reality:** once we choose an encoding circuit $\mathcal{E}(\boldsymbol{x})$, we already choose what kind of functions the model can represent.

$$|\phi(\boldsymbol{x})\rangle = \mathcal{E}(\boldsymbol{x})\,|0\rangle^{\otimes n}$$

The trainable circuit acts on $|\phi(\boldsymbol{x})\rangle$, not on the blank state $|0\rangle^{\otimes n}$.

# Basic Encodings at a Glance

| Encoding | Features | Qubits | Depth | Main issue |
|---|---|---|---|---|
| Basis | binary string | $d$ | low | no superposition |
| Angle | real values | $d$ | low | one qubit per feature |
| Phase | real values | $d$ | low | periodicity modulo $2\pi$ |
| Dense angle | real values | $\lceil d/2 \rceil$ | low | still product-state start |
| Amplitude | normalised vector | $\lceil \log_2 d \rceil$ | often high | state preparation cost |

## Practical default

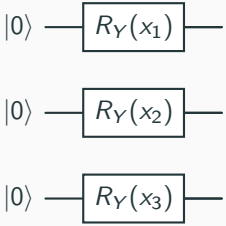
For near-term teaching and experiments, **angle encoding** is often the safest starting point: easy to explain, easy to implement, and hardware-friendly.

# Angle Encoding: The First Useful Workhorse

If a feature value is $x$, a simple encoding is

$$R_Y(x)\,|0\rangle = \cos\left(\frac{x}{2}\right)|0\rangle + \sin\left(\frac{x}{2}\right)|1\rangle\,.$$

- Each qubit starts in $|0\rangle$ by default.
- One feature controls one rotation angle.
- The encoded state is initially a **product state**.
- Entanglement is added later by the trainable or feature-map circuit.




## Why ML people like it

Expectation values become trigonometric functions of the data. After several layers or re-uploads, the circuit can represent richer Fourier-like structure.

# Amplitude Encoding: Qubit-Efficient but Not Free

For a normalised vector $\boldsymbol{x} = (x_0, \ldots, x_{2^n-1})$ we can write

$$\boldsymbol{x} \longmapsto \sum_{i=0}^{2^n-1} x_i \left| i \right\rangle .$$

- **Advantage:** $2^n$ amplitudes are stored in only $n$ qubits.
- **Catch:** generic state preparation can be expensive.
- Normalisation is mandatory because quantum states must satisfy $\|\psi\| = 1$.

Example:

$$(4, 8, 5, 0)/\sqrt{105}$$

fits into only 2 qubits:

$$\frac{1}{\sqrt{105}} \left(4 \left|00\right\rangle + 8 \left|01\right\rangle + 5 \left|10\right\rangle\right) .$$

**Takeaway**

Amplitude encoding saves qubits, but not necessarily runtime or noise budget.

# How Do We Encode an Image?

Start with a tiny image and flatten it into a feature vector:

$$I = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix},$$

$$x = (0, 1, 1, 0, 0, 0, 1, 0, \ldots, 1, 0) \in \mathbb{R}^{16}.$$

- In classical ML, this is already a legal feature vector.
- In QML, we must decide: which entries become angles, phases, or amplitudes?
- This choice determines both the **qubit count** and the **function class** available to the model.

| | | | |
|---|---|---|---|
| 0 | 1 | 1 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |

16 pixels = 16 raw features

## First conceptual step

Image encoding is a map from intensities to a vector, then from that vector into a quantum state.

Reference: IBM QML course (data encoding); Schuld & Petruccione, *ML with Quantum Computers*, 2nd ed. (Springer, 2021).

# Step 1: Turn the Image into Four Explicit Numbers

Take the same $4 \times 4$ image and split it into four $2 \times 2$ patches:

$$\left[\begin{array}{cc|cc} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ \hline 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{array}\right]$$

$$z_1 = \frac{0+1+0+0}{4} = 0.25, \qquad z_2 = \frac{1+0+1+0}{4} = 0.50,$$

$$z_3 = \frac{0+1+0+1}{4} = 0.50, \qquad z_4 = \frac{1+0+1+0}{4} = 0.50.$$

## Now every symbol is concrete

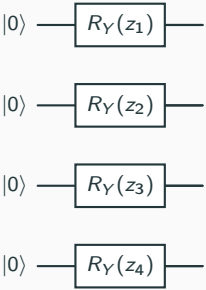
For this exact image, the four numbers are

$$\boldsymbol{z} = (0.25,\ 0.50,\ 0.50,\ 0.50).$$

# Step 2: Angle-Encode These Four Numbers

Map the four patch means $\boldsymbol{z} = (z_1, \ldots, z_4)$ to a four-qubit state (all qubits start in $|0\rangle$):

$$|\phi(\boldsymbol{z})\rangle = \big(R_Y(z_1) \otimes R_Y(z_2) \otimes R_Y(z_3) \otimes R_Y(z_4)\big)\,|0\rangle^{\otimes 4}\,.$$




- For our numbers: $R_Y(0.25)$ on the first wire, $R_Y(0.50)$ on the other three.
- Shallow encoding; the trade-off is that the image was already compressed classically into four scalars.

IBM QML course (angle / tensor-product encodings).

# Step 3: The Same Image with Amplitude Encoding

Instead of keeping only four averages, we can keep all sixteen pooled pixels and write

$$\boldsymbol{x} = (x_0, \ldots, x_{15}), \qquad \alpha = \sqrt{\sum_{i=0}^{15} x_i^2}, \qquad |\phi(\boldsymbol{x})\rangle = \frac{1}{\alpha} \sum_{i=0}^{15} x_i \, |i\rangle \, .$$

- Only 4 qubits are needed because $2^4 = 16$.
- The full image now lives in the amplitudes.
- However, we must prepare a very specific state, which is the hard part.

Compression in qubits:

$$16 \text{ pixels} \longrightarrow 4 \text{ qubits}$$

but not compression in circuit difficulty.

## Pedagogical punchline

Angle encoding is easy to run; amplitude encoding is easy to write down. These are not the same thing.

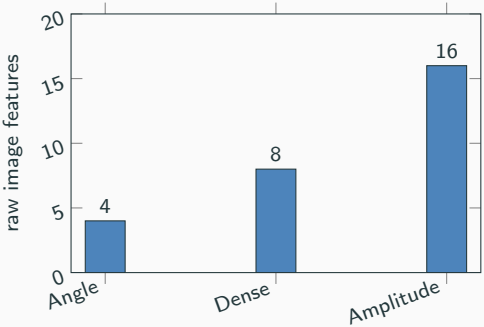
# Image Encoding Under a Fixed Qubit Budget

## Budget: 4 qubits

- Angle encoding: 4 raw features
- Dense angle encoding: 8 raw features
- Amplitude encoding: 16 amplitudes

## What people actually do

Resize, crop, patch, PCA, autoencoders, or classical convolution before quantum encoding.




## But do not over-read this plot

The amplitude bar counts **available amplitudes**, not free performance. State preparation and noise can completely erase this advantage on NISQ hardware.

# A More Image-Specific Idea: Patch Encoding

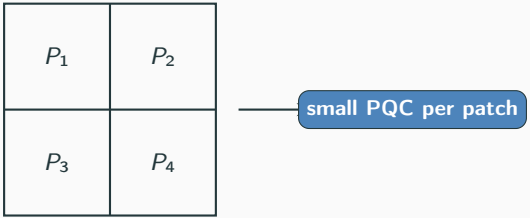
Images have strong **local structure**: neighbors are usually more related than distant pixels. Encode *patches*, not the whole image at once.

## Classical image intuition

- edges are local
- textures are local
- translation of a feature should not destroy it

## Quantum analogue

- split the image into small patches
- encode each patch into a small circuit
- combine patch outputs classically or with later quantum layers



# Patch encoding and classical vision

## Why this is image-specific

Same idea as local receptive fields in CNNs or local filters in classical image processing.

Reference: IBM Quantum Learning QML course; Qiskit Machine Learning tutorial: training on a real dataset; E. Grant et al., "Hierarchical quantum classifiers," arXiv:1804.03680 (2018); npj Quantum Inf. 4, 65 (2018).

# What Is a Feature Map, Precisely?

A **feature map** is a data-dependent unitary $U_\phi(\boldsymbol{x})$ that embeds classical data into Hilbert space:

$$|\phi(\boldsymbol{x})\rangle = U_\phi(\boldsymbol{x})\,|0\rangle^{\otimes n}.$$

- In a variational model, $U_\phi$ is the data-loading part of the network.
- In a kernel model, $U_\phi$ induces the similarity measure.
- A feature map is useful only if it makes relevant structure more visible after measurement.

## Important distinction

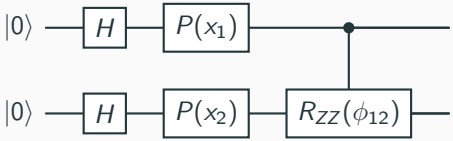
Basic encodings tell us *where* the numbers go. A feature map tells us *how the encoded data are transformed* before we compare or train on them.

# Example Feature Map: The ZZ Feature Map

The idea is to encode single-feature phases and pairwise feature interactions in the same circuit.

$$U_{ZZ}(\boldsymbol{x}) = \exp\left( i \sum_i x_i Z_i + i \sum_{i<j} \phi_{ij}(\boldsymbol{x}) Z_i Z_j \right) H^{\otimes n}, \qquad \phi_{ij}(\boldsymbol{x}) \propto (\pi - x_i)(\pi - x_j).$$

- First layer: place each qubit on the equator with $H$.
- Single-qubit phase terms encode each feature separately.
- $ZZ$ interactions encode pairwise correlations.




## Why it is interesting

Unlike pure angle encoding, the feature map already mixes features through entangling gates before any trainable layer is applied.

# Further reading: ZZ feature maps and quantum kernels

V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," Nature **567**, 209–212 (2019), DOI 10.1038/s41586-019-0980-2;

IBM Quantum Learning QML course (feature maps for kernels).

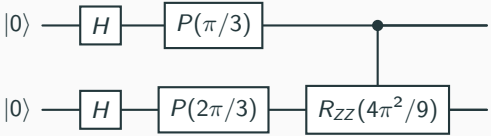
# Parametrized entanglement in ZZ-style maps

Many introductory circuits use **fixed** entanglers (e.g. CNOT). In ZZ feature maps, two-qubit interactions are usually **parametrized rotations** $R_{ZZ}(\phi_{ij}(\boldsymbol{x}))$: the angle sets how strongly that pair contributes at a given input.

- If $\phi_{ij}(\boldsymbol{x}) = 0$, that gate is the identity on the pair: no entangling effect from that edge for that $\boldsymbol{x}$.
- As $|\phi_{ij}|$ grows, the relative phase between computational components of opposite parity on qubits $i, j$ increases, which changes overlaps $\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle$ and the induced similarity measure.
- In the IBM ZZ map, $\phi_{ij}(\boldsymbol{x}) \propto (\pi - x_i)(\pi - x_j)$: features enter both **which** correlations are weak or strong and **how much** phase is imprinted.
- Independently of the angles, the **interaction pattern** (which pairs are wired) is a design choice; it fixes the entanglement *topology*, while the functions $\phi_{ij}$ fix the *strength* as a function of data.

# Concrete ZZ example: through the $P$ layers

Take $\boldsymbol{x} = (\pi/3,\, 2\pi/3)$ and $\phi_{12}(\boldsymbol{x}) = 2(\pi - x_1)(\pi - x_2) = 4\pi^2/9$ for the pairwise term. In the diagram, $P(x)$ means $e^{ixZ}$, i.e. the same $\exp\big(i\sum_i x_i Z_i\big)$ factor as in the definition of $U_{ZZ}$. Since $e^{ixZ}\,|b\rangle = e^{i(-1)^b x}\,|b\rangle$ for $b \in \{0,1\}$, acting on $|+\rangle^{\otimes 2}$ picks up one phase per bitstring; expanding the tensor product yields the four coefficients below.




Circuit read left to right; this slide stops *after* the two $P$ gates (before $R_{ZZ}$).

After $H^{\otimes 2}$, $|\psi_0\rangle = |+\rangle^{\otimes 2} = \frac{1}{2}\sum_{b_1,b_2\in\{0,1\}} |b_1 b_2\rangle$. Applying $e^{ix_1 Z} \otimes e^{ix_2 Z}$,

$$|\psi_1\rangle = \tfrac{1}{2}\big(e^{i(x_1+x_2)}\,|00\rangle + e^{i(x_1-x_2)}\,|01\rangle + e^{i(-x_1+x_2)}\,|10\rangle + e^{-i(x_1+x_2)}\,|11\rangle\big).$$

For this $\boldsymbol{x}$, $x_1 + x_2 = \pi$, $x_1 - x_2 = -\pi/3$, and $-x_1 + x_2 = \pi/3$, hence

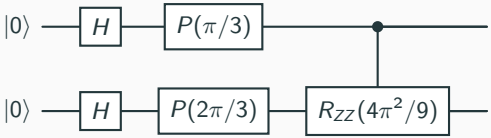
$$|\psi_1\rangle = \tfrac{1}{2}\big(-\,|00\rangle + e^{-i\pi/3}\,|01\rangle + e^{i\pi/3}\,|10\rangle - |11\rangle\big).$$

# Concrete ZZ example: the parametrized $ZZ$ layer

Continue with the same $\boldsymbol{x}$ and $\phi = 4\pi^2/9$. On computational kets, $Z_1 Z_2$ has eigenvalue $+1$ on $|00\rangle$ and $|11\rangle$ and $-1$ on $|01\rangle$ and $|10\rangle$, so $R_{ZZ}(\phi) = e^{i\phi Z_1 Z_2}$ multiplies the even-parity kets by $e^{i\phi}$ and the odd-parity kets by $e^{-i\phi}$. Applying this to $|\psi_1\rangle$ from the previous frame gives the encoded state

$$|\phi(\boldsymbol{x})\rangle = \tfrac{1}{2}\left(-e^{i\phi}\,|00\rangle + e^{-i(\phi+\pi/3)}\,|01\rangle + e^{i(\pi/3-\phi)}\,|10\rangle - e^{i\phi}\,|11\rangle\right).$$

Even versus odd parity now acquires an extra relative phase $2\phi$ between the two subspaces: that is the strength of the parametrized entanglement at this input. For a two-qubit state, a product state satisfies $c_{00}c_{11} = c_{01}c_{10}$ for amplitudes in the computational basis; here that equality fails for generic $\phi$, so $|\phi(\boldsymbol{x})\rangle$ is genuinely entangled.



# Contrast: A Product-State Map vs. an Entangling Map

## Z feature map

$$U_Z(\boldsymbol{x}) = \bigotimes_{i=1}^{n} P(x_i) H_i$$

- no entangling gate
- features remain separate
- classically easier to simulate

## ZZ feature map

$$U_{ZZ}(\boldsymbol{x}) = U_{ZZ\text{-int}}(\boldsymbol{x}) \left( \bigotimes_{i=1}^{n} P(x_i) H_i \right)$$

- entangling gate present
- pairwise correlations are encoded
- richer geometry in feature space

## Interpretation

Changing the feature map changes similarity: two points can be close in one map and farther in the other.

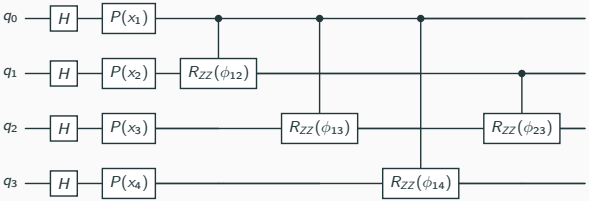


Reference: IBM QML course (Z / ZZ feature maps); Havlíček et al. (2019) (entangling quantum feature space / kernel view).

# What Changes When We Have 3, 4, or More Variables?

For $n$ inputs $(x_1, \ldots, x_n)$, a ZZ-style map typically has single terms $x_i Z_i$ and pairwise terms $\phi_{ij}(\boldsymbol{x}) Z_i Z_j$.

- $n$ single-feature terms
- $\binom{n}{2}$ pairwise terms
- $n = 3$: $3 + 3$ terms
- $n = 4$: $4 + 6$ terms




## Scaling intuition

The number of encoded correlations can grow quickly; feature-map design is a modeling choice, not only implementation.

# Feature-Map Philosophy: What Are We Trying to Encode?

## A good feature map should

- reflect the structure we believe matters
- separate useful similarity from useless similarity
- remain shallow enough to estimate reliably

## A bad feature map often

- encodes many interactions without motivation
- creates too much depth
- becomes hard to interpret or train

## Rule of thumb

Pick the simplest map that matches the correlations you care about:

- independent features $\rightarrow$ product-style maps
- pairwise correlations $\rightarrow$ ZZ-style maps
- higher-order structure $\rightarrow$ Pauli or learned maps

Reference: M. Schuld, R. Sweke & J. J. Meyer, arXiv:2008.08605; Phys. Rev. A **103**, 032430 (2021); Havlíček et al. (2019).

# Other Feature-Map Ideas

## Pauli feature maps

$$U(\boldsymbol{x}) = \exp\left( i \sum_S \phi_S(\boldsymbol{x}) \prod_{j \in S} \sigma_j \right)$$

with $\sigma_j \in \{X, Y, Z\}$.

This generalises $Z$- and $ZZ$-style maps.

## Exotic but useful ideas

- problem-informed graph feature maps
- Hamiltonian-inspired encodings
- data re-uploading maps
- learned feature maps inside hybrid models

## No free lunch

Making a feature map more exotic does not automatically improve learning. The extra structure must match the data and remain shallow enough to survive hardware noise.

Reference: M. Schuld, R. Sweke & J. J. Meyer, arXiv:2008.08605; Phys. Rev. A **103**, 032430 (2021).

# Feature Maps and Data Re-Uploading

With **data re-uploading**, we inject the same data more than once (two blocks shown; the pattern extends):

$$U(\boldsymbol{\theta}_2)\, U_\phi(\boldsymbol{x})\, U(\boldsymbol{\theta}_1)\, U_\phi(\boldsymbol{x})\, |0\rangle^{\otimes n}\,.$$

- Variational analogue of depth in a classical network.
- For angle encodings, re-uploading adds Fourier harmonics in the input.
- More expressivity without more qubits.

# Hardware-Efficient Circuits

# Hardware-efficient circuits on real devices

**Question.** What runs reliably on current hardware, and why do we use repetitive shallow layers?

- Two-qubit gates are still much noisier than single-qubit gates.
- Long-range couplings often require extra `SWAP`s after transpilation.
- Very expressive circuits can become hard to train and can suffer from **barren plateaus**.

**Design rule**

Keep the circuit as shallow as possible while still matching the structure of the task.

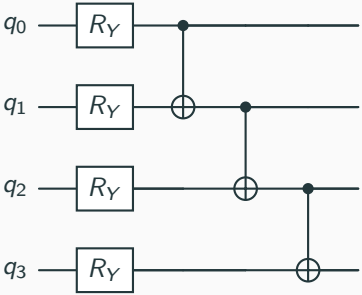
**Today's bias**

For QML on current hardware, **shallow and interpretable** often beats **maximally expressive on paper**.

# A Typical Hardware-Efficient Layer

$$W(\boldsymbol{\theta}) = \text{Entangle} \circ \bigotimes_{i=1}^{n} R_Z(\theta_i^{(1)}) R_Y(\theta_i^{(2)}).$$

- local rotations
- fixed entangling pattern
- repeated $L$ times




## In PennyLane

This is exactly the spirit of templates such as `StronglyEntanglingLayers`: easy to instantiate, easy to scale, but not automatically problem-aware.

# Hardware-Efficient vs. Problem-Inspired

| Question | Hardware-efficient | Problem-inspired |
|---|---|---|
| What decides the gates? | device constraints | domain structure |
| Why is it attractive? | shallow and portable | more meaningful search space |
| Main risk | weak inductive bias | too deep / too specialised |
| Typical use in Lecture 07 | hybrid QNNs | comparison point |

## For QML beginners

Start with a shallow hardware-efficient circuit. Add problem structure only when you know what pattern in the data you want to capture.

# Kernels

# Quantum kernels: why inner products matter

**Idea.** A quantum kernel is built from overlaps $\langle \phi(\boldsymbol{x}) | \phi(\boldsymbol{x}') \rangle$ between feature-map states; many algorithms only need these pairwise values.

Many learning rules (not only SVMs) need **pairwise comparisons** between data points in some feature space $\Phi(\boldsymbol{x})$.

- A **linear** classifier in feature space uses hyperplanes: scores are built from inner products $\langle \Phi(\boldsymbol{x}), \boldsymbol{w} \rangle$.
- The **representer theorem** (kernel setting): the optimal $\boldsymbol{w}$ often lies in the span of training features, so predictions depend on $K(\boldsymbol{x}, \boldsymbol{x}^{(i)}) = \langle \Phi(\boldsymbol{x}), \Phi(\boldsymbol{x}^{(i)}) \rangle$ rather than on $\Phi(\boldsymbol{x})$ explicitly.
- We therefore precompute (or estimate) a **kernel matrix** $K_{ij} = K(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)})$ and reuse the same $K$ at prediction time.

## Takeaway

The kernel is the interface: it encodes geometry and similarity after an implicit embedding.

# Kernels, Distances, and Similarity (Classical)

If $K(\boldsymbol{x}, \boldsymbol{y}) = \langle \Phi(\boldsymbol{x}), \Phi(\boldsymbol{y}) \rangle$ is a valid inner product kernel, then

$$\|\Phi(\boldsymbol{x}) - \Phi(\boldsymbol{y})\|^2 = K(\boldsymbol{x}, \boldsymbol{x}) + K(\boldsymbol{y}, \boldsymbol{y}) - 2K(\boldsymbol{x}, \boldsymbol{y}).$$

So **pairwise kernel values fix distances in feature space**, even when $\Phi$ is never formed.

**Linear kernel**
$K(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{y}$: cosine and Euclidean geometry in $\mathbb{R}^d$.

**RBF kernel**
$K(\boldsymbol{x}, \boldsymbol{y}) = \exp\left(-\gamma \|\boldsymbol{x} - \boldsymbol{y}\|^2\right)$: similarity decays with *input-space* Euclidean distance.

B. Schölkopf & A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (MIT Press, 2002).

# Classical Kernel First, in Plain Language

A kernel compares two data points $\boldsymbol{x}$ and $\boldsymbol{y}$ after an implicit mapping into a richer feature space:

$$K(\boldsymbol{x}, \boldsymbol{y}) = \langle \Phi(\boldsymbol{x}), \Phi(\boldsymbol{y}) \rangle.$$

- We often never write $\Phi(\boldsymbol{x})$ explicitly.
- The kernel value tells us how aligned two mapped points are.
- Support vector machines can work directly with the kernel matrix.

**Why kernels are powerful**

They let us use a nonlinear feature space while keeping the classical optimisation problem relatively simple.

# From Classical Kernel to Quantum Kernel

Choose a quantum feature map $U_\phi(\boldsymbol{x})$ and define

$$|\phi(\boldsymbol{x})\rangle = U_\phi(\boldsymbol{x})\,|0\rangle^{\otimes n}.$$

Then the quantum kernel is

$$K_Q(\boldsymbol{x},\boldsymbol{y}) = |\langle\phi(\boldsymbol{x}) \mid \phi(\boldsymbol{y})\rangle|^2 = \left|\langle 0|^{\otimes n}\, U_\phi^\dagger(\boldsymbol{y}) U_\phi(\boldsymbol{x})\,|0\rangle^{\otimes n}\right|^2.$$

## Interpretation

Two points are similar if their encoded quantum states have large overlap. The feature map decides what "similar" means.

# Quantum Kernel = Overlap of Quantum Feature States

- Prepare $U_\phi(\boldsymbol{x})\,|0\rangle^{\otimes n}$
- Apply $U_\phi^\dagger(\boldsymbol{y})$
- Measure the probability of the all-zero state

$$P(0\cdots 0) = K_Q(\boldsymbol{x}, \boldsymbol{y}).$$

# A Fully Explicit Toy Example: One-Qubit Angle Feature Map

Take the simplest feature map:

$$U_\phi(x) = R_Y(x), \qquad |\phi(x)\rangle = R_Y(x)\,|0\rangle\,.$$

For two data points $x$ and $y$,

$$K_Q(x,y) = |\langle 0|\, R_Y(-y) R_Y(x)\,|0\rangle|^2\,.$$

- One qubit
- Two uploaded data points
- Kernel entry = probability of measuring 0

# The Same Toy Example Worked Out Algebraically

Use the rotation-composition rule

$$R_Y(-y)R_Y(x) = R_Y(x-y).$$

Therefore

$$K_Q(x,y) = \left|\langle 0| \, R_Y(x-y) \, |0\rangle\right|^2 .$$

But

$$R_Y(\delta)\,|0\rangle = \cos\left(\frac{\delta}{2}\right)|0\rangle + \sin\left(\frac{\delta}{2}\right)|1\rangle ,$$

so

$$\boxed{K_Q(x,y) = \cos^2\left(\frac{x-y}{2}\right)}.$$

## Meaning

This kernel says that two points are similar when their angles are close on the Bloch sphere.

# Now Put Numbers In

Choose two concrete data points:

$$x = \frac{\pi}{3}, \qquad y = \frac{2\pi}{3}.$$

Then

$$x - y = -\frac{\pi}{3}, \qquad K_Q(x, y) = \cos^2\left(-\frac{\pi}{6}\right) = \cos^2\left(\frac{\pi}{6}\right) = \frac{3}{4}.$$

- If we run the overlap circuit many times, about 75% of the shots should return 0.
- That number becomes one entry in the kernel matrix.

| outcome | 0 | 1 |
|---|---|---|
| probability | 0.75 | 0.25 |

Same overlap logic as in Qiskit Machine Learning tutorial "Quantum kernel machine learning" and the IBM QML course; here reduced to a circuit you can check by hand.

# How the Kernel Matrix Is Actually Computed

Given training data $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(M)}\}$, we compute

$$K_{ij} = K_Q\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right).$$

1. Pick two data points $\boldsymbol{x}^{(i)}$ and $\boldsymbol{x}^{(j)}$.
2. Build the overlap circuit $U_\phi^\dagger(\boldsymbol{x}^{(j)})U_\phi(\boldsymbol{x}^{(i)})$.
3. Run the circuit many times.
4. Estimate $P(0\cdots0)$.
5. Store the result in the $i, j$ entry of the kernel matrix.

$$K = \begin{bmatrix} 1 & K_{12} & \cdots \\ K_{21} & 1 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Repeat for all pairs.

# From One Kernel Entry to the Full Matrix

For $M$ training points, we need all pairwise values

$$K_{ij} = K_Q\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right), \qquad i, j = 1, \ldots, M.$$

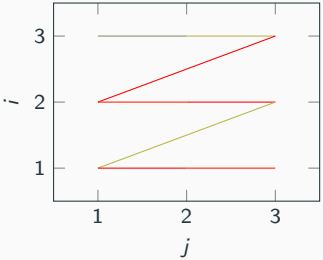
- Diagonal entries satisfy $K_{ii} = 1$ in the noise-free ideal case.
- Symmetry gives $K_{ij} = K_{ji}$, so half the matrix is redundant.
- In practice, we still estimate many overlap circuits.

$$K = \begin{bmatrix} 1 & K_{12} & K_{13} \\ K_{12} & 1 & K_{23} \\ K_{13} & K_{23} & 1 \end{bmatrix}$$

# A Tiny Visualization of Kernel Values

$$K = \begin{bmatrix} 1.00 & 0.91 & 0.18 \\ 0.91 & 1.00 & 0.24 \\ 0.18 & 0.24 & 1.00 \end{bmatrix}$$

- Points 1 and 2 are very similar after feature mapping.
- Point 3 is far from them in the induced Hilbert-space geometry.



# Classical Kernel SVM vs. Quantum Kernel (QSVM)

| | **Classical kernel SVM** | **QSVM (kernel estimator)** |
|---|---|---|
| Feature map | $\Phi$ implicit; pick $K$ (e.g. RBF) | $U_\phi(\boldsymbol{x})\,\lvert 0\rangle$; $K_Q$ from overlaps |
| Kernel entry | Closed form from $\boldsymbol{x}, \boldsymbol{y}$ | Estimate $\lvert \langle 0\rvert\, U_\phi^\dagger(\boldsymbol{y})\, U_\phi(\boldsymbol{x})\,\lvert 0\rangle \rvert^2$ (shots or sim.) |
| Geometry | RKHS induced by $K$ | Geometry from chosen $U_\phi$ (entanglement matters) |
| Cost driver | $O(M^2)$ matrix for $M$ points | Same matrix size; each entry is a quantum workload |

## Same classical head

Both use a **classical** convex program on a Gram matrix; the difference is only how $K_{ij}$ is produced.

# QSVM Workflow in Three Layers

1. **Quantum layer:** compute the train and test kernel matrices.
2. **Classical SVM layer:** solve the margin-maximisation problem using the precomputed kernel.
3. **Prediction layer:** classify a new point with the support vectors and learned coefficients.

# QSVM: What Happens After the Kernel Is Built?

The quantum part computes the kernel matrix. The classification itself is still classical:

$$f(\boldsymbol{x}) = \operatorname{sign}\left(\sum_{i=1}^{M} \alpha_i y_i K_Q(\boldsymbol{x}^{(i)}, \boldsymbol{x}) + b\right).$$

- $y_i \in \{-1, +1\}$ are class labels.
- $\alpha_i$ and $b$ come from the classical SVM optimisation.
- The quantum computer supplies the similarity values $K_Q$.

**So QSVM is hybrid**

Quantum: kernel estimation
Classical: margin maximisation and prediction

Havlíček et al. (2019); Qiskit ML: quantum kernel tutorial; `qiskit_machine_learning: QSVC` (quantum kernel + SVM).

# Why Quantum Kernels Attracted So Much Interest

- Quantum states live in a Hilbert space whose dimension grows exponentially with qubit number.
- Entangling feature maps can generate similarity measures that may be hard to reproduce classically.
- There are rigorous results showing advantage for **special carefully constructed tasks**.

**Important caveat**

High-dimensional quantum states are **not** by themselves a guarantee of useful learning. The feature map must match the structure of the data, and the loading/measurement costs still matter.

# When Quantum Kernels May Help — and When Not

## Potentially promising

- data with nontrivial correlations
- structured feature maps inspired by domain knowledge
- moderate-size tasks where kernel estimation is still feasible

## Usually not promising

- data already solved well by cheap classical kernels
- very large datasets requiring huge kernel matrices
- encodings whose circuit depth kills the signal

## Practical mindset

Think of quantum kernels as **new similarity functions to explore**, not as a universal replacement for classical kernels.

# Hybrid Models

# Hybrid models: PennyLane, PyTorch, and optional Catalyst

**Course setup:** hybrid quantum–classical training with **PennyLane** and **PyTorch**.

PennyLane **Catalyst** adds `@qjit`: JIT/AOT compilation of hybrid programs, often with `lightning.qubit`. The supported training stack is **JAX-native**: `jax.numpy` arrays, `catalyst.grad` for derivatives, and optimisers such as **Optax** for compiled update steps.

**Course practice** uses **PennyLane** + **PyTorch** only; Catalyst is optional context if you later work in a JAX workflow.

**Scope**

Compilation speeds up a **given** model; it does not fix poor encodings or barren plateaus.

PennyLane Catalyst documentation; PennyLane demo: QJIT / VQE with Catalyst.

# Where the Quantum Layer Sits in a Hybrid Model

## Why this is attractive

The quantum block does not need to do everything. It can provide a compact nonlinear feature map, while the classical part handles scalable optimisation and flexible output layers.

# One Explicit Forward Pass

Suppose one data point has four classical features:

$$\boldsymbol{x} = (x_1, x_2, x_3, x_4) \in \mathbb{R}^4.$$

We encode it on four qubits, measure four expectation values, then pass them to a classical head:

$$\boldsymbol{x} \xrightarrow{\text{quantum encoding + PQC}} \boldsymbol{q} = \begin{bmatrix} \langle Z_0 \rangle \\ \langle Z_1 \rangle \\ \langle Z_2 \rangle \\ \langle Z_3 \rangle \end{bmatrix} \in \mathbb{R}^4 \xrightarrow{W\boldsymbol{q}+b} \text{logit} \xrightarrow{\sigma} \hat{y}.$$

## Interpretation

The quantum layer behaves like a learnable feature extractor. The classical layer turns those features into a final prediction.

# Readout: From Quantum Outputs to a Prediction (1/2)

After the circuit you choose **observables**, estimate **expectation values** (shots or exact simulation), and pack them into a real vector $\boldsymbol{q} \in \mathbb{R}^m$ (e.g. $\langle Z_i \rangle$, $\langle X_i \rangle$, or Pauli-string expectations).

## Direct readout heads (no extra neural net yet)

- **Regression:** $\hat{y} = w^\top \boldsymbol{q} + b$, or a single $\langle O \rangle$ if $m = 1$.
- **Binary:** $\ell = w^\top \boldsymbol{q} + b$, $\hat{p} = \sigma(\ell)$; or threshold a chosen $\langle Z \rangle$.
- **Multi-class ($C$ classes):** $\boldsymbol{\ell} = W\boldsymbol{q} + \boldsymbol{b}$ with $W \in \mathbb{R}^{C \times m}$, $\widehat{\boldsymbol{p}} = \mathrm{softmax}(\boldsymbol{\ell})$.
- **Multi-label:** $L$ logits with independent sigmoids (or one small head per label).

# Readout: Classical Layers and Losses (2/2)

## When $\boldsymbol{q}$ feeds a classical network

Use $\boldsymbol{q}$ like any feature vector: dense layers, nonlinearities, then softmax or sigmoid heads as in ordinary deep learning. The quantum circuit is only the **feature extractor**.

## What you must match in code

- Define **which** observables form $\boldsymbol{q}$, and the **shot budget** if not exact.
- Choose the map $\boldsymbol{q} \mapsto \hat{y}$ (linear head or larger net).
- Pick a **loss** consistent with the task: e.g. MSE for regression; for classification, cross-entropy with softmax logits, or binary cross-entropy on sigmoid outputs.

# Minimal PennyLane + PyTorch Pattern

```python
import pennylane as qml
import torch

dev = qml.device("default.qubit", wires=4)

@qml.qnode(dev, interface="torch")
def qlayer(x, theta):
    for i in range(4):
        qml.RY(x[i], wires=i)
    qml.StronglyEntanglingLayers(theta, wires=range(4))
    return [qml.expval(qml.Z(i)) for i in range(4)]
```

- The quantum node returns differentiable outputs.
- PyTorch treats those outputs like features from any other layer.
- Gradients can flow through both the classical and quantum parameters.

PennyLane interfaces / `QNode`; `TorchLayer`; IBM QML course.

# What Are the Shapes of the Objects?

## Mini-batch view

If the batch size is $B$, then

$$X \in \mathbb{R}^{B\times 4}.$$

Each row is one data point.

## Quantum output

After the quantum layer,

$$Q \in \mathbb{R}^{B\times 4}.$$

Each row contains four expectation values.

## Classical head

For one dense layer,

$$W \in \mathbb{R}^{1\times 4}, \qquad b \in \mathbb{R}.$$

The logits are

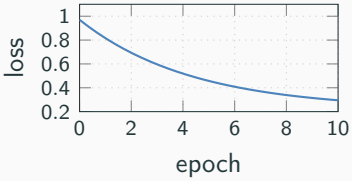
$$\ell = QW^\top + b \in \mathbb{R}^{B\times 1}.$$

## Why this helps

Students can now see that the quantum layer is not mysterious: it simply outputs a feature vector.

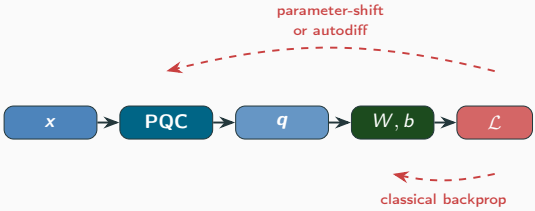
# Training Loop: What Happens Behind the Scenes?

1. Encode the mini-batch into circuit parameters.
2. Run the quantum layer and measure expectation values.
3. Feed those values into a classical head.
4. Compute the loss.
5. Backpropagate through the classical part and the quantum part.
6. Update parameters with Adam or another optimiser.




Plot is illustrative: the practical notebook will produce the real curves.

# How Do Gradients Flow Through the Hybrid Model?




PQC: $U_\phi(\boldsymbol{x})$ and $U(\boldsymbol{\theta})$; $\boldsymbol{q}$ collects $\langle Z_i \rangle$.

- $W, b$: ordinary backpropagation.
- $\boldsymbol{\theta}$: parameter-shift or other circuit gradients.
- One loss ties classical and quantum updates together.

# Three Related Models to Compare

### Pure PQC classifier

input $\rightarrow$ PQC $\rightarrow$ one measured observable $\rightarrow$ class

### PQC + dense layer

input $\rightarrow$ PQC $\rightarrow$ several observables $\rightarrow$ dense layer

### PQC + dense + ReLU

input $\rightarrow$ PQC $\rightarrow$ dense layer $\rightarrow$ ReLU $\rightarrow$ class

### Philosophy

Moving from left to right shifts more modeling power into the classical head while keeping the quantum circuit as a feature generator.

# Nonlinearity

# Nonlinearity: where does it come from?

The unitary part of a circuit is linear in the state vector, but the **learned map** from data to predictions need not be.

- The evolution of a closed quantum state is linear: $|\psi\rangle \mapsto U|\psi\rangle$.
- Machine learning predictions use **measurements**, probabilities, losses, and often classical post-processing.
- Therefore the overall map from input data to prediction can be **nonlinear**, even if the quantum state evolution is unitary.

Example: if $f(x) = \langle Z \rangle$ and the encoded state contains $\cos(x)$ and $\sin(x)$, then after a classical logistic or ReLU layer the final predictor is already nonlinear in $x$.

# Classical Analogy: Where Is the Nonlinearity?

## Classical neural net

$$x \xrightarrow{Wx+b} h \xrightarrow{\sigma(h)} \text{nonlinear feature}$$

The activation
sigma breaks linearity.

## Hybrid quantum model

$$\boldsymbol{x} \xrightarrow{U_\phi(\boldsymbol{x}),U(\theta)} \langle O \rangle \xrightarrow{\text{dense/ReLU}} \text{prediction}$$

Measurement and classical post-processing play a similar role.

## Key point

If the quantum block is followed by a classical nonlinear head, the full model is nonlinear in exactly the same practical sense as a classical network.

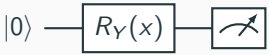
# Measurement Already Produces a Nonlinear Function of Data

Consider one qubit with angle encoding:

$$|\phi(x)\rangle = R_Y(x)\,|0\rangle = \cos\left(\frac{x}{2}\right)|0\rangle + \sin\left(\frac{x}{2}\right)|1\rangle\,.$$

Measure $Z$:

$$f(x) = \langle\phi(x)|\,Z\,|\phi(x)\rangle = \cos(x).$$




- The circuit itself is unitary.
- The expectation value is already a nonlinear function of $x$.
- Adding more layers creates richer trigonometric combinations.

# Four Useful Sources of Effective Nonlinearity

1. **Measurement + classical loss**: probabilities are nonlinear functions of amplitudes.
2. **Data re-uploading**: more harmonics, richer input dependence.
3. **Hybrid heads**: quantum outputs fed into dense layers, ReLU, softmax.
4. **Open-system effects / partial trace**: tracing out ancillas gives an effectively non-unitary map on the visible subsystem.

## Course emphasis

In practice, the easiest and most useful nonlinearities to demonstrate are **re-uploading** and **classical heads after a quantum layer**.

# Data Re-Uploading as a Nonlinearity Booster

**Pattern.** Encode the same scalar $x$ repeatedly, alternating with **trainable** single-qubit blocks $U(\theta_j)$:

$$U(\theta_L)R_Y(x)\cdots U(\theta_2)R_Y(x)U(\theta_1)R_Y(x)\,|0\rangle\,.$$

Because $R_Y(x) = \exp(-\mathrm{i}xY/2)$, each $x$ enters as a phase in an exponential. After expanding products of such exponentials and using the trainable $U(\theta_j)$ to change basis between insertions, expectations of local Pauli observables (e.g. $\langle Z\rangle$) become **finite trigonometric sums** in $x$: linear combinations of $\cos(kx)$ and $\sin(kx)$ with bounded $|k|$ (often called a Fourier / harmonic picture in the QML literature).

**Not the same as "$x, x^2, x^3$".** Applying $R_Y(x)$ *back-to-back with nothing in between* composes as $R_Y(x)^m = R_Y(mx)$: you only **rescale the angle**. That is **not** the re-uploading trick above, which needs **interleaved** $U(\theta_j)$ to mix the $x$-dependent rotations and generate many harmonics.

# Data re-uploading: harmonics and references

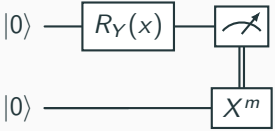
Compared to a single $R_Y(x)$ layer on the same number of qubits:

- More $x$-blocks (with trainable layers between) $\Rightarrow$ typically more harmonics in $x$ for fixed observables.
- Same qubit count, richer dependence on $x$ than a single $R_Y(x)$ layer.

Pérez-Salinas et al., arXiv:1907.02085; Quantum 4, 226 (2020).

# Mid-Circuit Measurement and Conditional Branching

Mid-circuit measurement creates a genuinely measurement-driven branch in the computation:



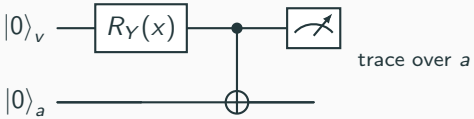

- Measure the first qubit and store the classical outcome $m \in \{0, 1\}$.
- Apply a gate on another qubit *conditioned* on $m$.
- This resembles an "if-then" branch in a classical program and can create effective nonlinear behaviour in the data-to-output map.

IBM Quantum / Qiskit documentation (search: dynamic circuits, mid-circuit measurement); PennyLane: measurements and mid-circuit operations. Advanced extension, not the default first hybrid model.

# Ancilla + Partial Trace: Another Route

Introduce an ancilla, entangle it with the data qubit, then **discard** (trace out) the ancilla:

$$\rho_{\mathrm{vis}}(x) = \mathrm{Tr}_a\big(U_{va}(x)\,\rho_0\,U_{va}^{\dagger}(x)\big)\,.$$




(ancilla not measured in the diagram)

- Joint state on $v \otimes a$ can be **pure** after $U_{va}$.
- $\rho_{\mathrm{vis}}$ is the **reduced state**: in general **mixed**, even if the joint state was pure.
- Ignoring $a$ is not the same as applying a unitary on $v$ alone.

M. A. Nielsen & I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniv. ed. (Cambridge Univ. Press) (reduced states, partial trace); M. M. Wilde, *Quantum Information Theory* (Cambridge Univ. Press); related lecture notes arXiv:1106.1445.

# Why Tracing Out Can Look "Nonlinear" in $x$

**Pure vs. mixed:** A density matrix $\rho$ is pure iff $\rho = |\psi\rangle\langle\psi|$ (rank one). A mixed state is a convex combination of pure states; $\rho_{\mathrm{vis}}$ after tracing out an entangled ancilla is typically mixed.

**Map on inputs:** For fixed gate layout, $x \mapsto \rho_{\mathrm{vis}}(x)$ is built from a **linear** map on the *joint* system followed by a **linear** partial trace. The result is still generally a **nonlinear** function of $x$ in the sense of matrix entries (same phenomenon as $|\psi(x)\rangle$ depending nonlinearly on $x$ before trace).

**Connection to outputs:** When you then compute expectations or classical post-processing from $\rho_{\mathrm{vis}}(x)$, you can obtain **nonlinear** functions of $x$ that are not realisable as a single unitary on $v$ *without* an ancilla or measurement.

## Intuition

Tracing out discards correlations; the remaining state carries less information in a way that resembles **noise or coarse graining**, not a reversible rotation on $v$ alone.

See also CPTP / quantum channels (open systems): Wilde, arXiv:1106.1445 (textbook-level treatment beyond this slide).

# Advanced Mechanisms Worth Knowing

## Quantum-side ideas

- repeat-until-success constructions
- mid-circuit measurement and feedback
- ancilla qubits followed by tracing out
- QCNN pooling by measurement / coarse graining

## Architecture-side ideas

- Strongly Entangling Layers
- quantum layer followed by dense layer and ReLU
- multiple quantum layers with classical skip connections
- hybrid feature extractor + classical classifier

## Pedagogical point

Not every mechanism needs a full derivation in one lecture. For practice, it is better to master one clean hybrid example than to collect many buzzwords.

# Summary

# Summary: closing remarks

1. **QML today is mostly hybrid.**
2. **Encoding is part of the model**, not an afterthought.
3. **Image encoding is constrained by qubit budget and depth**; resizing and compression are essential.
4. **Hardware-efficient circuits** matter because shallow circuits preserve signal and are easier to train.
5. **Quantum kernels** are similarity measures induced by quantum feature maps.
6. **Nonlinearity** appears through measurement, re-uploading, and hybrid quantum-classical architectures.

**Practice session**

We will implement image encoding, estimate a small quantum kernel matrix, and train a PennyLane + PyTorch hybrid model.